



The markdown is the source of truth. Everything this tool writes is derived and safe to delete.

The shape of it





Every command is `atlas <name>` in a shell and `/atlas:<name>` in Claude Code. ■ shell only □ Claude Code only

Starting out

<code>build</code>	adopt this repository and generate the site — the one command that does a first run end to end
<code>help</code>	this list (also what a bare atlas prints)
<code>config [--json]</code>	the resolved configuration — the file merged over the defaults, which is what runs
<code>version</code>	which build is answering, where it lives, and whether it is behind

Seeing where things stand

□ <code>status</code>	the corpus, the rot and the one next action
<code>health [--verbose]</code>	report rot signals; exit 1 if any blocking signal fires
<code>tasks [word]</code>	the planning document with progress bars; a word filters it
<code>design [--scaffold]</code>	which design documents exist, and which are scaffolds still owing their substance
□ <code>dashboard</code>	build, serve and hand back a URL
<code>product</code>	every sibling repository under one directory at once — which have adopted this tool and which have not — for the case where the session is running from a directory that is not a repository, and every other surface here is therefore blank

Working on a change

<code>branch [type slug]</code>	branch state, or create type/short-slug carrying your changes
<code>plan [slug]</code>	the route for the change in your tree — branch, item, version; --apply cuts the branch
<code>changes [--json]</code>	what is uncommitted and which documents cite it
<code>diff <file></code>	that file's diff — uncommitted, else across the branch
□ <code>review</code>	what this diff breaks or fixes, before the commit

Picking up cold

<code>state [--json]</code>	what a resuming session reads first: where you are, what was recorded
<code>handoff</code>	the derived half of a handoff, for the half only a person can write
<code>note <kind> "<text>"</code>	record one decision, trap or blocker — the only command here that writes to the repository

Digging into history

<code>contrib [--json]</code>	who did what, from git history alone
<code>ownership [--json]</code>	bus factor per area — how many people have ever touched it
<code>surviving [--json]</code>	whose written lines are still in the tree, by git blame rather than by commit
<code>worklog [--day D]</code>	write today's entry from git and the plan; --stdout to see it without writing
<code>sessions [--out F]</code>	how sessions went — turns, interruptions, friction, rework
<code>tokens [--out FILE]</code>	token accounting from local session transcripts — opt-in, never published
<code>--snapshot</code>	append the counts-only day rollout to .atlas/tokens.jsonl (needs tokens.snapshot)

What git history says, and nothing else here reads

<code>git-insights [sect]</code>	what git history says that nothing else here reads — strictly read-only sections: hotspots, coupling, branches, cadence, hygiene, change
----------------------------------	--

What git history says, and nothing else here reads (cont.)

□ <code>git-hotspots</code>	the files change keeps returning to, what changes together, and who knows each area
□ <code>git-history</code>	cadence and commit hygiene
□ <code>git-branch</code>	branch state and the branch inventory
<code>git-tree</code>	the branch topology — what was cut from what, drawn as a tree, with every origin marked as the inference it is
□ <code>git-status</code>	where you are, what is uncommitted, and what this change touches
□ <code>git-diff <path></code>	one file, with the history around it

Stopping and picking back up

<code>pause [--dry-run]</code>	checkpoint every agent worktree to a wip/agent-* ref before the session ends — writes git refs
<code>resume</code>	print the re-spawn plan for a paused session — branch, worktree, checkpoint
<code>stop [--force]</code>	clear session state and agent worktrees; every branch and checkpoint survives

Publishing

<code>caps</code>	which features the host has on — makes one network request
<code>community [--write]</code>	issue and PR scaffolding for the features that came back on
<code>publish</code>	wiki, pages or a single file, and never without confirmation
<code>--target wiki</code>	GitHub Wiki — flattened markdown, links rewritten, drift-guarded
<code>--target pages</code>	the full site to a gh-pages branch (dashboard + deck survive)
<code>--target export</code>	one self-contained HTML file (--page dashboard index health)
<code>--ci</code>	(GitLab, pages) write the .gitlab-ci.yml job instead — Pages is an artifact there
□ <code>artifact</code>	the dashboard, health report or index to claude.ai as a shareable page — it leaves this machine, with every local-only panel stripped and the export asserting that itself

Agent surfaces

<code>mcp</code>	serve the corpus over MCP on stdio — read-only, no dependency
<code>--status</code>	what a client would connect to, where it is registered, what is running
<code>ask <task></code>	one structured answer for a program; exit 1 on findings, 2 if it could not run
<code>prompt [--out FILE]</code>	a system prompt assembled from this repository's own rules and state

Command line only

■ <code>init [--plan PATH]</code>	write project-atlas.config.json, detecting the layout and the plan document
■ <code>scan [--json]</code>	build and summarise the index
■ <code>contention [branch...]</code>	what a fan-out will collide on: files more than one branch touches, plan ids defined twice, and the next free id. --base REF; exit 1 on a duplicate id only
■ <code>spec --gate</code>	the commit gate: refuse a staged change whose message names no plan item
■ <code>serve</code>	build, then run the live dashboard detached and open it
<code>--status --stop</code>	what is running here end it now
<code>--list --launcher</code>	every atlas dashboard on this machine write a page linking them all
■ <code>watch [--serve]</code>	rebuild on change; --serve hosts it live at http://127.0.0.1:4173
■ <code>all</code>	scan + health + build